Lab 09 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — Scripts versus Compose

Answer. (1) **State**: `up` compares the file with the existing containers and recreates only what changed; a script restarts everything or fails on "name already in use". (2) **Order and waiting**: `depends_on` + `healthcheck` wait until a service is ready; a script strings `sleep` calls together. (3) **Scope**: `down` knows exactly what the project created (containers, network, pod) and removes it without touching anything else; a clean-up script always forgets something, or removes too much. Add the merging of environments (`-f`, override) and readability for a newcomer.

Why. Compose is **declarative**: it describes a target state and computes the difference. A script is **imperative**: it describes actions, without knowing the expected result.

Nuance. A script stays useful for what Compose does not cover (creating an engine secret, initialising a file). The two coexist: the script calls Compose.

Example.

```
podman compose up -d && podman compose up -d     # the second time: nothing to do, no container recreated
```

Question 2 — `.env` does not enter the container

Answer. `.env` is read by **Compose** to substitute the `${…}` in the YAML file (`${API_TAG:-1.0}`); it is passed to no container. `APP_MESSAGE` belonged in the service's `environment:`, or in a file referenced by `env_file:` (`api.env`), which is passed to the container as `--env-file`.

Why. Two mechanisms, two moments: substitution happens when the YAML is read, the environment is built when the container is created.

Nuance. A **shell** variable is also used for substitution, before `.env`. And `podman compose config` shows the substituted YAML: if `${APP_MESSAGE}` does not appear in it, `.env` has no effect at all.

Example.

```
podman compose config | grep -c 'hello from dotenv'     # 0: the variable is used nowhere
podman compose exec api env | grep APP_MESSAGE           # the value from api.env
```

Question 3 — Short and long `depends_on`

Answer. Short form: `db` is **created and started** before `api`, but `api` leaves immediately; PostgreSQL does not accept connections yet, Spring Boot fails (`Connection refused`, `HikariPool … Exception during pool initialization`) and the container exits with an error code — without `restart`, it stays `Exited`. Long form: `api` is started only once `db`'s healthcheck has passed (`healthy`); Spring Boot finds the database, everything is `Up`. The condition requires a `healthcheck`: without one, `db` has no health state, and podman-compose refuses or waits forever.

Why. "Started" is a state of the container; "ready" is a state of the application, which only a healthcheck can observe.

Nuance. With `restart: unless-stopped`, the short form "eventually works" after a few restarts: it is fragile and hides the problem. Our fake API does not connect to the database; the effect shows in the **order** (`podman compose ps` shows `api` started five seconds after `db`), not in a failure.

Example.

```
podman compose ps     # db  Up 11 seconds (healthy)  /  api  Up 6 seconds (healthy)  /  web  Up 1 second
```

Question 4 — `curl` missing from the image

Answer. `eclipse-temurin:21-jre-alpine` does not contain `curl`: every run of the healthcheck fails (`exec: "curl": executable file not found`), and after `retries` failures the state becomes `unhealthy` — `web`, which waits for `service_healthy`, never starts. Diagnosis: `podman inspect stack_api_1 --format '{{json .State.Health.Log}}'`, or `podman healthcheck run stack_api_1`. Fix: use `wget` (BusyBox, present in Alpine): `["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]`.

Why. The healthcheck runs **inside** the container, with the image's binaries: a missing command is a failure like any other.

Nuance. Distroless images have neither `curl` nor `wget` nor a shell: the healthcheck must then be a bundled binary (or, in Kubernetes, an HTTP probe run by the kubelet, with no command).

Example.

```
podman compose exec api wget -qO- http://localhost:8080/actuator/health     # {"status":"UP"}: the command exists
podman compose exec api curl                                                # executable file not found
```

Question 5 — Named volume or bind mount for the database

Answer. `pgdata:`: Compose creates the `stack_pgdata` volume, Podman manages its directory, the files belong to the translated UID of `postgres` (`100069` on WSL), `down` keeps it, `down -v` deletes it. `./data`: you create the folder (or Podman creates it, owned by you), the files inside carry the same translated UID, so `ls ./data` answers `Permission denied` and `git status` trips over it; `down -v` **does not touch it**. For the database: the named volume — backed up with `podman volume export` (lab 06), outside the repository, with no permission surprises.

Why. Compose manages only what it declares; a host path is yours, a volume is its own.

Nuance. The bind mount is the right choice for **configuration** (`nginx-proxy.conf`, `index.html`): versioned, readable, editable live.

Example.

```
podman compose down && podman volume ls | grep stack_pgdata     # still there
podman compose down -v && podman volume ls | grep -c stack_pgdata   # 0
```

Question 6 — The ghost project

Answer. Without `name:`, the project is named after the folder: `stack` on one side, `stack-v2` on the other. The second `up` created `stack-v2_default`, `stack-v2_pgdata` (empty) and `stack-v2_db_1`: a second stack, which does not see the first one's volume. Nothing is lost: `podman volume ls` shows both volumes. Two safeguards: `name: stack` in the file, or `podman compose -p stack up -d` (or `COMPOSE_PROJECT_NAME`).

Why. The project name is the prefix of every resource; changing the name means changing worlds.

Nuance. Two projects side by side are sometimes **intended** (two branches, two versions) — provided they do not publish the same ports.

Example.

```
podman volume ls --format '{{.Name}}'     # stack_pgdata  stack-v2_pgdata
podman compose -p stack-v2 down -v         # remove the ghost, keep the original
```

Question 7 — Two answers for one container

Answer. `podman exec` tells the truth: it prints the environment **of the process**, `prod`. `podman compose exec` re-reads the **default** Compose files (`compose.yaml` + `compose.override.yaml`, since no `-f` is given), derives the service's environment from them and injects it into the `exec` command: `dev` comes from the override, not from the container. To avoid it: give `exec` the same `-f` as `up`, set `COMPOSE_FILE=compose.yaml:compose.prod.yaml` in the shell, or use `podman exec`.

Why. podman-compose passes `--env` to `podman exec` to reproduce the declared environment; it does not know which files the container was created with.

Nuance. Docker Compose behaves the same way for file selection: `-f` is never remembered. `COMPOSE_FILE` is the convention of both tools.

Example.

```
podman compose -f compose.yaml -f compose.prod.yaml exec api env | grep APP_PROFILE   # prod
```

Question 8 — `restart: unless-stopped` without a daemon

Answer. With Docker, the daemon starts with the machine and relaunches the containers whose policy requires it: the stack comes back on its own. With rootless Podman, there is **no daemon**: after `wsl --shutdown`, nobody re-reads the restart policies; the containers stay `Exited` until a `podman compose up -d` or `podman start`. The policy only acts **while Podman is running** (application crash → relaunch). For an automatic start: **systemd** units generated by Quadlet (lab 10), or `podman compose up -d` in a `systemd --user` unit with *linger*.

Why. `--restart` is executed by `conmon` and Podman for a live container; no permanent process exists to act at system boot.

Nuance. `podman generate systemd` (deprecated) and Quadlet produce units that systemd does relaunch at boot — that is Podman's answer to `restart: always`.

Example.

```
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'   # unless-stopped
wsl --shutdown   # (from Windows) then: podman ps -a → Exited
```

Question 9 — Two APIs behind nginx

Answer. Both containers carry the **alias** `api` on the `stack_default` network; the internal DNS returns both addresses, and nginx (which resolves at startup) takes whichever it gets — in practice an approximate spread, not real load balancing. `--scale web=2` fails because both instances would publish `18080` on the host: port conflict. With podman-compose, `down` only knows instance 1: you must remove `stack_api_2` by name (`podman rm -f -t 0 stack_api_2`) before `down`, otherwise `db` cannot be removed (dependency).

Why. Scaling multiplies containers under one service name; the real distribution is the job of a load balancer (Kubernetes Service, lab 13).

Nuance. Docker Compose handles scale-down and `down`. It is one of the concrete limitations of podman-compose to be aware of.

Example.

```
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address   # two addresses
podman rm -f -t 0 stack_api_2 && podman compose down -v
```

Question 10 — "name already in use"

Answer. A `stack_db_1` already exists, left behind by an incomplete `down` (for instance after a `--scale`, question 9) or by a manual `podman run --name`. `up` creates a fresh container from the file and cannot "adopt" a container that lacks its project labels or whose configuration differs. Repair: `podman ps -a` to look, `podman rm -f -t 0 stack_db_1` (the **volume** `stack_pgdata` is untouched), then `up -d`.

Why. Container names are unique per engine; Compose never silently removes a container it does not recognise.

Nuance. `podman compose down` first is the good habit; if it fails itself, named clean-up is the only way — never `podman rm -af` on a shared machine.

Example.

```
podman ps -a --filter name=stack_ --format '{{.Names}} {{.Status}}'
podman rm -f -t 0 stack_db_1 && podman compose up -d && podman volume ls | grep stack_pgdata
```

Question 11 — Secret through a file

Answer. With `secrets:` + `_FILE`: `podman compose config` shows `file: ./db_password.txt` (the path), `podman inspect` shows `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`, and Git sees only the YAML file — **provided** `db_password.txt` is in `.gitignore`. With `environment: POSTGRES_PASSWORD: S3cr3t`: the value is in the YAML (hence in Git), in `config`, in `inspect`. `db_password.txt`: outside Git, `600` permissions, one per environment, or replaced by an engine secret with `external: true` (`podman secret create` beforehand, lab 08).

Why. Compose mounts the file at `/run/secrets/<name>`; the value passes through no metadata.

Nuance. Docker Compose outside Swarm does exactly the same (a file mount); `external: true` refers to a Swarm secret there, to a `podman secret` with Podman.

Example.

```
podman compose config | grep -A1 'db_password:'          # file: ./db_password.txt
podman inspect stack_db_1 --format '{{json .Config.Env}}' | grep -c S3cr3t   # 0
```

Question 12 — Kubernetes equivalents

Answer. Service with `image` → **Deployment** (and its Pod); `ports` → **Service** (ClusterIP/NodePort) and, for the outside, **Ingress**; `pgdata` volume → **PersistentVolumeClaim** (and StatefulSet for a database); `healthcheck` → **readiness/liveness probes**; `depends_on` → **no direct equivalent**: Kubernetes does not manage start order, each Pod retries until its dependencies answer, and the *readiness probe* keeps traffic away until then; implicit network → the **cluster** network and CoreDNS; `env_file` → **ConfigMap**; `secrets` → **Secret**.

Why. Kubernetes is designed for components that fall over and come back: it replaces "wait" with "retry, and do not route until ready".

Nuance. *Init containers* can wait for a dependency, but that is a workaround, not the model.

Example.

```
podman compose config > stack.yaml   # starting point for kompose, or for writing the lab 12 manifests
```

Question 13 — Compose in production

Answer. Reasonable for an internal, low-traffic application, with no availability requirement, on a VM run by a small team — which describes many real applications. What Compose does not provide: (1) **high availability** (one VM, one single point of failure); (2) **zero-downtime updates** (an `up` recreates the container, the service drops for a few seconds); (3) **distribution across machines** and autoscaling; (4) **automatic restart** under rootless with no daemon, and encrypted secret management. The signal: as soon as you write scripts to do what Compose does not (failover, replication, rolling update), it is time to move to an orchestrator.

Why. Compose is a tool for **local composition**, not a cluster scheduler.

Nuance. Between the two sit simple options: Quadlet + systemd for restarts, a load balancer in front of two Compose VMs, or single-node k3s (lab 11), which brings the Kubernetes mechanisms without the size of a cluster.

Example.

```
podman compose -f compose.yaml -f compose.prod.yaml up -d && podman compose ps   # one VM, three containers, one single point of failure
```